

100 Days of Azure DevOps

Day 67 Handout — Secret scan, SCA, SAST — publish SARIF; fail on high; a ignored scan is décor

7 - Security, Compliance & Governance · Personal lab only · Educational content · Not a sales pitch

Architecture A — three scans and one result format

Scan	When it runs	What 'done' is
Secret scanning	On the repo / PR (gitleaks, ADO secret scanning, GitHub push protection)	Build fails if a key-shaped string lands. Day 64 is wasted otherwise
SCA (dependencies)	On restore (pip-audit, npm audit, NuGet)	Lockfile + fail on high/critical. A lockfile without a scan is a list
SAST	On source (language analyzer if you have one)	Findings next to the commit. If you have no task, document the gap
SARIF	Static Analysis Results Interchange Format — JSON the tools emit	Publish the file (e.g. CodeAnalysisLogs). Humans and portals both ingest it
Collector products	Microsoft Defender for DevOps / Microsoft Security DevOps task can gather SARIF	A concept: one publisher of results. Not a marketplace shopping lab

Architecture B — fail criteria before you install a brand

Finding	Default gate	Warn-only when
Secret in repo	Fail. Always	Never. Rotate if it already shipped
Critical / high CVE in a dep you ship	Fail if the tool can exit non-zero	First run baseline — time-box, then tighten. Not forever
Medium / noise	Warn in the summary	Silent continueOnError — that is how kitchens burn politely
Tool missing	Script + open-source scanner in Docker	Skipping because the Microsoft task needs an extra install click
Where results go	Pipeline summary + SARIF artifact	Line 4000 of a log nobody opens

Architecture C — pitfalls that make a green build look 'secure'

Pitfall	What you see	Fix
Scan not in PR	Main is clean; feature branches leak secrets	Same job on pull request validation
continueOnError: true	Always green, SARIF ignored	Remove it. Fail the job
Product tour instead of a gate	Three extensions, zero fail criteria	One secret scan that can fail. Brands are optional collectors
Dummy DemoSecret committed	gitleaks may or may not fire on 'not-a-real-password'	Do not commit it to test. Use a fake AWS AKIA pattern in a branch you will drop if you must test fail
SARIF unpublished	Tool ran, Azure DevOps shows nothing	Write results.sarif; publish artifact. Ingestion is the point of the format

One-liner to remember

Shift-left security means finding the fire in the kitchen, not on the evening news. SARIF is the receipt format; the gate is a non-zero exit on secrets and critical CVEs.

Step-by-step lab (personal Azure / Azure DevOps only)

Personal azure-100-labs. Microsoft-hosted ubuntu-latest. No employer scanners. No product-spam install-all.

- 1 Add a secret-scan step (gitleaks in Docker is enough). --exit-code 1. Publish a SARIF file as an artifact if the tool can emit it.
- 2 Add a dependency scan on restore for whatever the sample app uses (npm audit --audit-level=high, pip-audit, etc.). Decide fail vs warn in a markdown file — not in your head.
- 3 Optional: Microsoft Security DevOps / Defender for DevOps as a SARIF collector — only if it is already easy. The concept is 'tools → SARIF → pipeline'. Do not spend the lab on a marketplace tour.
- 4 Run on a branch. If you need to prove fail, use a throwaway file with a clearly fake credential pattern and delete that commit. Do not commit Day 64's vault value.
- 5 Confirm results are visible without opening raw logs (summary, artifact, or annotations). continueOnError must be off for the secret step.
- 6 Write docs/scan-day67.md: tools used, fail criteria, SARIF yes/no. Tomorrow the Security stage wraps this.

Starter YAML — secret scan to SARIF, fail closed

```
pool:
  vmImage: ubuntu-latest
steps:
- bash: |
    docker run --rm -v "$(pwd):/repo" ghcr.io/gitleaks/gitleaks:latest detect \
      --source=/repo --report-format sarif --report-path /repo/gitleaks.sarif --exit-code 1
    displayName: Secret scan (fail on findings)
- task: PublishBuildArtifacts@1
  inputs:
    PathToPublish: gitleaks.sarif
    ArtifactName: CodeAnalysisLogs
  condition: always()
- bash: npm audit --audit-level=high
  displayName: SCA — fail on high (Node sample)
# SARIF is a format. Defender for DevOps is one optional collector, not the homework.
```

Done checklist

- Secret scan can fail the job
- Knows SAST vs SCA vs secrets as three jobs, not one brand
- SARIF treated as the interchange format, not a product name
- Fail criteria written; continueOnError not used as a gate

Tomorrow — Day 68

DevSecOps shift-left — Security stage before Deploy, not a final boss